



Pipeline de Indicadores Macroeconomicos Brasileiros

Relatorio Tecnico - Projeto Final

AUTORES

Andre Henrique Pacheco Alves

Enrico Comassetto Di Gioia

Enzo de Moraes Godoy

Maria Luiza Oliveira de Souza

Luzivania Jesus Bonfim

1. Sumário Executivo

Este documento descreve o projeto **Pipeline de Indicadores Macroeconômicos Brasileiros**, desenvolvido como trabalho final da disciplina *Data Integration* (ESPM 2026.1). O objetivo central é construir um pipeline ETL completo — da extração à carga — que consolide dados heterogêneos do Banco Central do Brasil (BCB) e do Conselho Monetário Nacional (CMN) num Data Warehouse relacional, orquestrado pelo Apache Airflow em ambiente Docker.

O sistema extrai automaticamente três indicadores macroeconômicos de alta relevância — **taxa SELIC**, **IPCA mensal** e **câmbio USD/BRL** — da API pública do BCB (formato JSON), além de uma série histórica de metas de inflação via arquivo CSV estático. Após validações de qualidade, os dados são normalizados para um modelo dimensional (star schema) e carregados via upsert idempotente no Postgres, com agendamento semanal via Airflow.

Indicador	Valor
Séries extraídas	3 (API BCB) + 1 CSV
Cobertura histórica	24 meses (configurável)
Registros de fato	~2.500 (diário + mensal)
Tabelas no DW	4 (3 dimensões + 1 fato)
Tasks na DAG	5 (init_db → extract → validate → transform → load)
Testes automatizados	24 casos (pytest)
Tecnologias	Python 3.11, Airflow 2.9, PostgreSQL 15, Docker

Tabela 1 — Indicadores quantitativos do projeto

2. Objetivos

O projeto tem como objetivo principal demonstrar a implementação de um pipeline de integração de dados de ponta a ponta, abrangendo todas as etapas do processo ETL com boas práticas de engenharia de dados. Os objetivos específicos são:

- Extrair dados de **fontes heterogêneas** (REST API JSON e arquivo CSV) de forma automatizada
- Aplicar **validações de qualidade** nos dados brutos antes da transformação
- Normalizar os dados para um **modelo dimensional** (star schema) adequado para análises
- Carregar os dados via **upsert idempotente**, permitindo re-execuções sem duplicatas
- Orquestrar todo o fluxo com **Apache Airflow**, incluindo agendamento e retries automáticos
- Garantir **reprodutibilidade** total via Docker Compose com um único comando de inicialização
- Cobrir a lógica ETL com **testes automatizados** (pytest)

3. Arquitetura da Solução

3.1 Visão Geral do Fluxo

O pipeline segue a arquitetura clássica ETL dividida em cinco tasks sequenciais, orquestradas pela DAG macro_pipeline no Apache Airflow. Os dados fluem das fontes externas até o Data Warehouse em Postgres, passando por etapas de validação e transformação intermediárias:

Task	Módulo	Responsabilidade	Saída
init_db	SQLAlchemy	Cria schema dw e tabelas (IF NOT EXISTS)	Schema Postgres
extract	extract.py	GET API BCB + leitura CSV	raw_*.json, raw_metas.parquet
validate	validate.py	Nulos, ranges, duplicatas pré-transform	Exceção ou OK
transform	transform.py	Normaliza para modelo dimensional	dim_*.parquet, fact_*.parquet
load	load.py	Upsert nas tabelas do DW	DW atualizado

Tabela 2 — Tasks da DAG e suas responsabilidades

3.2 Representação ASCII do Pipeline



3.3 Decisões de Arquitetura

■ Arquivos intermediários em vez de XCom

Com 24 meses de cotações diárias (~1.500 registros × 3 séries), o payload excede o limite padrão de 48 KB do XCom do Airflow. Gravar em data/tmp/ (volume Docker compartilhado) resolve o problema e mantém os dados brutos acessíveis para debug.

■ Upsert idempotente (ON CONFLICT DO UPDATE)

Toda inserção usa INSERT ... ON CONFLICT, tornando o pipeline idempotente. Re-execuções e correções não geram duplicatas. Essencial no Airflow, onde retries são comuns.

■ DDL em arquivo SQL externo

O arquivo create_tables.sql é executado a cada run via init_db. Isso mantém separação clara entre infraestrutura (SQL) e lógica de negócio (Python), além de permitir versionamento independente do schema.

■ LocalExecutor em vez de CeleryExecutor

Suficiente para o volume e número de workers desta carga. Reduz drasticamente a complexidade do docker-compose, eliminando Redis e workers extras.

■ PostgreSQL único com dois bancos

Os bancos airflow (metadados) e dw (Data Warehouse) convivem no mesmo container. Justificativa: simplicidade operacional para ambiente de desenvolvimento/avaliação. Em produção, seriam instâncias separadas com SLAs e backups distintos.

4. Fontes de Dados

4.1 API do Banco Central do Brasil

A API pública do BCB (api.bcb.gov.br) fornece dados históricos de séries temporais econômicas. É acessada via HTTP GET sem autenticação, retornando JSON semi-estruturado no formato [{"data": "DD/MM/YYYY", "valor": "X.XX"}].

Chave Interna	Código BCB	Nome	Periodicidade	Unidade
selic_diaria	432	SELIC — Taxa Over	Diária	% a.a.
ipca_mensal	433	IPCA — Variação Mensal	Mensal	%
cambio_usd	1	Câmbio USD/BRL (venda)	Diária	R\$

Tabela 3 — Séries extraídas da API BCB

4.2 CSV de Metas de Inflação (CMN)

Arquivo CSV estático com a série histórica de metas de inflação definidas pelo Conselho Monetário Nacional. Cobre o período 2015–2025 e inclui meta central, bandas superior/inferior e inflação efetivamente realizada (IPCA acumulado do ano). Esta fonte complementa a API BCB, sendo **estruturalmente heterogênea** (CSV vs JSON) e com **granularidade diferente** (anual vs diária/mensal).

Coluna	Tipo	Descrição
ano	Integer	Ano-referência da meta
meta_inflacao	Float	Meta central do CMN (%)
banda_superior	Float	Limite superior da banda (%)
banda_inferior	Float	Limite inferior da banda (%)
inflacao_realizada	Float	IPCA acumulado realizado (%)

Tabela 4 — Estrutura do CSV de metas de inflação

5. Modelo Dimensional (Star Schema)

O DW adota um modelo dimensional (star schema) com uma tabela fato central e três dimensões. O workload é analítico — agregações temporais e cruzamentos entre séries — o que torna o star schema mais eficiente que um esquema relacional normalizado, reduzindo o número de JOINS nas queries de valor.

5.1 Tabela Fato

Coluna	Tipo	Descrição
id	SERIAL PK	Surrogate key gerada automaticamente
data	DATE FK	Referência para dim_tempo
indicador_id	INTEGER FK	Referência para dim_indicador
valor	NUMERIC(12,6)	Valor do indicador na data

Tabela 5 — fact_serie_temporal (grão: 1 valor por indicador por data)

5.2 Dimensões

Dimensão	PK	Cardinalidade aprox.	Notas
dim_tempo	data	~730 linhas	Atributos: dia, mês, ano, trimestre, semestre, nome_mês
dim_indicador	id	3 linhas	Metadados estáticos das séries (código BCB, unidade, etc.)
dim_meta_inflacao	ano	11 linhas	Dimensão degenerada — JOIN pelo ano, não por FK do fato

Tabela 6 — Dimensões do DW

5.3 Relacionamentos

```
dim_indicador ■■■■
    ■■■■ fact_serie_temporal ■■■■ dim_tempo
dim_meta_inflacao (auxiliar — JOIN por ano, sem FK direta no fato)
```

Nota sobre dim_meta_inflacao: esta dimensão é referenciada analiticamente por ano (JOIN entre dim_tempo.ano e dim_meta_inflacao.ano), não por chave estrangeira direta no fato. Isso ocorre porque o grão das metas é anual enquanto o fato é diário/mensal — uma FK convencional geraria fan-out.

6. Validações de Qualidade de Dados

O módulo `validate.py` implementa quatro regras de qualidade aplicadas antes da transformação (sobre dados brutos) e durante a transformação (integridade referencial):

#	Regra	Colunas alvo	Comportamento ao violar
1	Nulos em campos críticos	data, valor	Levanta <code>ValidationError</code> — aborta pipeline
2	Duplicatas na chave	(data, indicador_id)	Loga aviso, remove (keep=first)
3	Range de valores	valor por série	Loga aviso, remove outliers do fato
4	Integridade referencial	fact.data → dim_tempo.data	Levanta <code>ValidationError</code> — aborta pipeline

Tabela 7 — Regras de validação implementadas

6.1 Ranges Esperados por Série

Série	Mínimo	Máximo	Justificativa
selic_diaria	0,0%	50,0%	SELIC nunca foi negativa; 50% cobre choques históricos extremos
ipca_mensal	-5,0%	30,0%	Deflação pontual é possível; 30% cobre hiperinflação residual
cambio_usd	R\$ 0,50	R\$ 20,00	Margem ampla para cenários de crise cambial severa

Tabela 8 — Ranges de validação por série

7. Detalhamento das Etapas ETL

7.1 Extração (extract.py)

A função `extract_bcb()` itera sobre o dicionário `SERIES_CONFIG` e realiza chamadas HTTP GET à API do BCB com parâmetros de data (janela de 24 meses configurável via variável de ambiente `BCB_MONTHS_HISTORY`). Cada série é persistida individualmente como JSON em `data/tmp/raw_{chave}.json`.

A função `extract_csv()` lê o arquivo `metas_inflacao.csv` via `pandas` e o converte para Parquet (`raw_metas.parquet`), preservando tipos sem necessidade de re-parsear strings em etapas posteriores.

7.2 Validação (validate.py)

Executada como task independente antes da transformação, a validação opera sobre os arquivos brutos (`.json` e `.parquet`). Isso garante que problemas de qualidade sejam detectados o mais cedo possível no pipeline, evitando processamento desnecessário e dados corrompidos no DW.

7.3 Transformação (transform.py)

A função `transform_all()` orquestra a construção de todas as entidades dimensionais:

- **dim_indicador:** construída estaticamente a partir do dicionário `SERIES_CONFIG` — não depende de dados externos
- **Fatos brutos:** cada registro BCB é parseado com `_parse_bcb_record()`, que converte datas DD/MM/YYYY e valores string para float, descartando registros inválidos
- **dim_tempo:** gerada dinamicamente a partir do conjunto de datas únicas presentes nos fatos
- **Integridade referencial:** verificada antes da persistência (toda data do fato deve existir na `dim_tempo`)
- **dim_meta_inflacao:** lida do Parquet, enriquecida com a coluna derivada `bateu_meta` (booleano)

7.4 Carga (load.py)

Os quatro loaders seguem a mesma estratégia: leitura do Parquet correspondente, iteração linha a linha e execução de `INSERT ... ON CONFLICT DO UPDATE` via `SQLAlchemy`. A ordem de carga respeita as constraints de FK: dimensões primeiro, fato por último.

```
-- Exemplo: upsert do fato
INSERT INTO dw.fact_serie_temporal (data, indicador_id, valor)
VALUES (:data, :indicador_id, :valor)
ON CONFLICT (data, indicador_id) DO UPDATE
    SET valor = EXCLUDED.valor
```


8. Queries Analíticas

Três queries SQL foram desenvolvidas para demonstrar o valor analítico do DW, cada uma respondendo a uma pergunta de negócio relevante:

Query 1 — Evolução da SELIC (média mensal)

"Como a taxa básica de juros variou mês a mês nos últimos 24 meses?"

```
SELECT t.ano, t.mes, t.nome_mes,  
       ROUND(AVG(f.valor)::numeric, 4) AS selic_media_mensal_pct  
FROM dw.fact_serie_temporal f  
JOIN dw.dim_tempo t ON f.data = t.data  
JOIN dw.dim_indicador i ON f.indicador_id = i.id  
WHERE i.chave = 'selic_diaria'  
GROUP BY t.ano, t.mes, t.nome_mes  
ORDER BY t.ano, t.mes;
```

Query 2 — IPCA acumulado × Meta CMN

"O Brasil cumpriu a meta de inflação em cada ano?"

```
SELECT t.ano,  
       ROUND(SUM(f.valor)::numeric, 2) AS ipca_acumulado_pct,  
       m.meta_inflacao, m.banda_inferior, m.banda_superior, m.bateu_meta  
FROM dw.fact_serie_temporal f  
JOIN dw.dim_tempo t ON f.data = t.data  
JOIN dw.dim_indicador i ON f.indicador_id = i.id  
LEFT JOIN dw.dim_meta_inflacao m ON t.ano = m.ano  
WHERE i.chave = 'ipca_mensal'  
GROUP BY t.ano, m.meta_inflacao, m.banda_inferior, m.banda_superior, m.bateu_meta  
ORDER BY t.ano;
```

Query 3 — Câmbio USD/BRL por trimestre

"Qual foi a volatilidade cambial em cada trimestre?"

```
SELECT t.ano, t.trimestre,  
       ROUND(AVG(f.valor)::numeric, 4) AS cambio_medio,  
       ROUND(MAX(f.valor)::numeric, 4) AS cambio_max,  
       ROUND(MIN(f.valor)::numeric, 4) AS cambio_min,  
       ROUND((MAX(f.valor) - MIN(f.valor))::numeric, 4) AS amplitude  
FROM dw.fact_serie_temporal f  
JOIN dw.dim_tempo t ON f.data = t.data  
JOIN dw.dim_indicador i ON f.indicador_id = i.id  
WHERE i.chave = 'cambio_usd'  
GROUP BY t.ano, t.trimestre ORDER BY t.ano, t.trimestre;
```

9. Testes Automatizados

O projeto cobre a lógica ETL com **24 casos de teste** organizados em 9 classes, implementados com pytest. Os testes focam nas funções de transformação e validação, que concentram as regras de negócio críticas:

Classe de Teste	Função testada	Casos	Cobertura principal
TestParseBcbRecord	_parse_bcb_record()	6	Parsing de datas, valores, campos ausentes, inválidos
TestBuildDimTempo	build_dim_tempo()	4	Colunas, semestre, trimestre, deduplicação
TestBuildDimIndicador	build_dim_indicador()	2	Cardinalidade e chaves esperadas
TestTransformMetas	_transform_metas()	3	bateu_meta, colunas de saída, valores nulos
TestCheckNulls	check_nulls()	2	Levanta exceção e cenário OK
TestCheckDuplicates	check_duplicates()	2	Remoção e cenário sem duplicatas
TestCheckRange	check_range()	2	Remoção de outliers e modo strict
TestCheckReferentialIntegrity	check_referential_integrity()	2	Cenário OK e detecção de órfãos

Tabela 9 — Suite de testes automatizados

Como executar os testes

```
# Instalar dependências
pip install -r requirements.txt

# Rodar todos os testes
pytest tests/ -v

# Com relatório de cobertura
pytest tests/ -v --tb=short
```

10. Guia de Execução

10.1 Pré-requisitos

- Docker 24+
- Docker Compose v2
- Git

10.2 Inicialização

```
# 1. Clonar o repositório
git clone <URL_DO_REPO> && cd <pasta>

# 2. Configurar variáveis de ambiente
cp .env.example .env

# 3. Subir todos os serviços
docker compose up --build -d

# 4. Aguardar inicialização (~60s) e acessar o Airflow
# http://localhost:8080 (admin / admin)
```

10.3 Execução da DAG

Após o login no Airflow Webserver (localhost:8080), localize a DAG macro_pipeline, ative-a pelo toggle e dispare manualmente via *Trigger DAG*. O pipeline completo leva aproximadamente 2–3 minutos dependendo da latência da API BCB.

10.4 Verificação dos Dados

```
# Conectar ao banco dw
docker exec -it <container_postgres> psql -U dw -d dw

-- Contar registros no fato
SELECT COUNT(*) FROM dw.fact_serie_temporal;

-- Verificar dimensões
SELECT * FROM dw.dim_indicador;

-- Executar queries analíticas
\i /sql/queries.sql
```

10.5 Variáveis de Ambiente

Variável	Padrão	Descrição
BCB_MONTHS_HISTORY	24	Janela histórica em meses para extração BCB
METAS_CSV_PATH	/opt/airflow/data/	Caminho do CSV de metas dentro do container
DW_CONN_STR	postgresql+...	Connection string do Data Warehouse
AIRFLOW_WWW_USER_*	admin/admin	Credenciais do Airflow Webserver

AIRFLOW__CORE__FERNET_KEY(gerado)	Chave de criptografia de conexões Airflow
-----------------------------------	---

Tabela 10 — Variáveis de ambiente configuráveis

11. Stack Tecnológica

Camada	Tecnologia	Versão	Função
Linguagem	Python	3.11	Implementação de todos os módulos ETL
Orquestração	Apache Airflow	2.9	DAG, agendamento, retries, monitoramento
Banco de Dados	PostgreSQL	15	Armazenamento do DW e metadados Airflow
Container	Docker + Docker Compose	24 / v2	Ambiente reproduzível, isolamento de serviços
Data Wrangling	pandas	2.2.2	Transformação e manipulação de DataFrames
Serialização	pyarrow	16.1.0	Leitura/escrita de Parquet (artefatos intermediários)
HTTP Client	requests	2.32.3	Chamadas à API REST do BCB
DB Driver	psycopg2-binary	2.9.9	Adapter Postgres para SQLAlchemy
ORM/Engine	SQLAlchemy	(airflow)	Execução de SQL e upserts no DW
Testes	pytest	8.2.2	Suite de testes automatizados

Tabela 11 — Stack tecnológica completa

12. Uso de Inteligência Artificial

O desenvolvimento deste projeto contou com o suporte de **Claude (Anthropic)** nas seguintes frentes:

- Geração e revisão da arquitetura do pipeline e do modelo dimensional
- Scaffolding inicial dos módulos ETL (extract, transform, validate, load)
- Escrita do DDL (create_tables.sql) e das queries analíticas
- Estruturação e redação da documentação técnica (README, ARCHITECTURE.md)
- Revisão de boas práticas de engenharia de dados (idempotência, upsert, qualidade)

Todo o código foi revisado, testado e adaptado pelo grupo. O uso de IA foi tratado como ferramenta de produtividade e revisão, não como substituto do entendimento técnico.